



Lab Manual

Practical and Skills Development

CERTIFICATE

THE ASSIGNMENT ENTERED IN THIS REPORT HAVE BEEN
SATISFACTORILY PERFORMED BY

Registration No : 25BAI11429
Name of Student : Abhijeet Wadnere
Course Name : Introduction to Problem Solving and Programming
Course Code : CSE1021
School Name : SCAI
Slot : B11+B12+B13
Class ID : BL2025260100796
Semester : FALL 2025/26

Course Faculty Name : Dr. Hemraj S. Lamkuche

Signature:

Practical Index

S. No.	Title of Practical	Date of Submission	Signature of Faculty
1	Write a function called <code>euler_phi(n)</code> that calculates Euler's Totient Function, $q(n)$. This function counts the number of integers up to n that are coprime with n (i.e., numbers k for which $\gcd(n, k) = 1$).		
2	Write a function called <code>mobius(n)</code> that calculates the Möbius function, $u(n)$. The function is defined as: $\mu(n) = 1$ if n is a square-free positive integer with an even number of prime factors. $\mu(n) = -1$ if n is a square-free positive integer with an odd number of prime factors. $\mu(n) = 0$ if n has a squared prime factor.		
3	Write a function called <code>divisor_sum(n)</code> that calculates the sum of all positive divisors of n (including 1 and n itself). This is often denoted by $\sigma(n)$.		
4	Write a function called <code>prime_pi(n)</code> that approximates the prime-counting function, $\pi(n)$. This function returns the number of prime numbers less than or equal to n .		
5	Write a function called <code>legendre_symbol(a, p)</code> that calculates the Legendre symbol (a/p) , which is a useful function in quadratic reciprocity. It is defined for an odd prime p and an integer a not divisible by p as: $(a/p) = 1$ if a is a quadratic residue modulo p (i.e., there exists an integer x such that $x^2 = a \pmod{p}$).		

	$\left(\frac{a}{p}\right) = -1$ if a is a quadratic non-residue modulo p. You can calculate it using Euler's criterion: $\left(\frac{a}{p}\right) = a^{((p-1)/2)} \bmod p$.		
6	Write a function <code>factorial(n)</code> that calculates the factorial of a non-negative integer n ($n!$).		
7	Write a function <code>is_palindrome(n)</code> that checks if a number reads the same forwards and backwards.		
8	Write a function <code>mean_of_digits(n)</code> that returns the average of all digits in a number.		
9	Write a function <code>digital_root(n)</code> that repeatedly sums the digits of a number until a single digit is obtained.		
10	Write a function <code>is_abundant(n)</code> that returns True if the sum of proper divisors of n is greater than n .		
11	Write a function <code>is_deficient(n)</code> that returns True if the sum of proper divisors of n is less than n .		
12	Write a function for harshad number <code>is_harshad(n)</code> that checks if a number is divisible by the sum of its digits.		
13	Write a function <code>is_automorphic(n)</code> that checks if a number's square ends with the number itself.		
14	Write a function <code>is_pronic(n)</code> that checks if a number is the product of two consecutive integers.		
15	Write a function <code>prime_factors(n)</code> that returns the list of prime factors of a number.		
16	Write a function <code>count_distinct_prime_factors(n)</code> that returns how many unique prime factors a number has.		
17	Write a function <code>is_prime_power(n)</code> that checks if a number can be expressed as p^k where p is prime and $k \geq 1$.		

18	Write a function <code>is_mersenne_prime(p)</code> that checks if $2^p - 1$ is a prime number (given that p is prime).		
19	Write a function <code>twin_primes(limit)</code> that generates all twin prime pairs up to a given limit.		
20	Write a function <code>Number of Divisors (d(n)) count_divisors(n)</code> that returns how many positive divisors a number has.		
21	Write a function <code>aliquot_sum(n)</code> that returns the sum of all proper divisors of n (divisors less than n).		
22	Write a function <code>are_amicable(a, b)</code> that checks if two numbers are amicable (sum of proper divisors of a equals b and vice versa).		
23	Write a function <code>multiplicative_persistence(n)</code> that counts how many steps until a number's digits multiply to a single digit.		
24	Write a function <code>is_highly_composite(n)</code> that checks if a number has more divisors than any smaller number.		
25	Write a function for Modular Exponentiation <code>mod_exp(base, exponent, modulus)</code> that efficiently calculates $(base^{exponent}) \% modulus$.		
26	Write a function <code>Modular Multiplicative Inverse mod_inverse(a, m)</code> that finds the number x such that $(a * x) = 1 \bmod m$.		
27	Write a function <code>chinese Remainder Theorem Solver crt(remainders, moduli)</code> that solves a system of congruences $x = r_i \bmod m_i$.		
28	Write a function <code>Quadratic Residue Check is_quadratic_residue(a, p)</code> that checks if $x^2 = a \bmod p$ has a solution.		

29	Write a function <code>order_mod(a, n)</code> that finds the smallest positive integer <code>k</code> such that $ak \equiv 1 \pmod n$.		
30	Write a function <code>Fibonacci Prime Check is_fibonacci_prime(n)</code> that checks if a number is both Fibonacci and prime.		
31	Write a function <code>Lucas Numbers Generator lucas_sequence(n)</code> that generates the first <code>n</code> Lucas numbers (similar to Fibonacci but starts with 2, 1).		
32	Write a function for Perfect Powers Check <code>is_perfect_power(n)</code> that checks if a number can be expressed as ab where $a > 0$ and $b > 1$.		
33	Write a function <code>Collatz Sequence Length collatz_length(n)</code> that returns the number of steps for <code>n</code> to reach 1 in the Collatz conjecture.		
34	Write a function <code>Polygonal Numbers polygonal_number(s, n)</code> that returns the <code>n</code> -th <code>s</code> -gonal number.		
35	Write a function <code>Carmichael Number Check is_carmichael(n)</code> that checks if a composite number <code>n</code> satisfies $a^{n-1} \equiv 1 \pmod n$ for all <code>a</code> coprime to <code>n</code> .		
36	Implement the probabilistic Miller-Rabin test <code>is_prime_miller_rabin(n, k)</code> with <code>k</code> rounds.		
37	Implement <code>pollard_rho(n)</code> for integer factorization using Pollard's rho algorithm.		
38	Write a function <code>zeta_approx(s, terms)</code> that approximates the Riemann zeta function $\zeta(s)$ using the first 'terms' of the series.		
39	Write a function <code>Partition Function p(n) partition_function(n)</code> that calculates the number of distinct ways to write <code>n</code> as a sum of positive integers.		

Practical No: 1

Date: _____

TITLE:

Week 1, Question 1

AIM/OBJECTIVE(s):

Write a function called `euler_phi(n)` that calculates Euler's Totient Function, $q(n)$. This function counts the number of integers up to n that are coprime with n (i.e., numbers k for which $\gcd(n, k) = 1$).

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import math
```

```
import time
```

```
def get_gcd(a, b):
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def euler_phi(n):
    count = 0
    for k in range(1, n + 1):
        if get_gcd(n, k) == 1:
            count += 1
    return count

def euler_phi_efficient(n):
    result = n
    p = 2
    while p * p <= n:
        if n % p == 0:
            while n % p == 0:
                n = n // p
            result = result - (result // p)
        p += 1
    if n > 1:
        result = result - (result // n)
    return result

if __name__ == "__main__":
    number = 10000

    print(f'Calculating phi({number})...')

    start_time = time.time()
    answer = euler_phi(number)
    end_time = time.time()
```

```
print(f'Result (Counting Method): {answer}')  
print(f'Time Taken: {end_time - start_time:.5f} seconds')  
print("-" * 30)
```

```
start_time = time.time()  
answer_fast = euler_phi_efficient(number)  
end_time = time.time()
```

```
print(f'Result (Efficient Method): {answer_fast}')  
print(f'Time Taken: {end_time - start_time:.5f} seconds')
```

RESULTS ACHIEVED:

Calculating phi(10000)...

Result (Counting Method): 4000

Time Taken: 0.00358 seconds

Result (Efficient Method): 4000

Time Taken: 0.00001 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 2**Date:** _____**TITLE:**

Week 1, Question 2

AIM/OBJECTIVE(s):

Write a function called mobius(n) that calculates the Möbius function, $\mu(n)$. The function is defined as:

- **$\mu(n) = 1$ if n is a square-free positive integer with an even number of prime factors.**
- **$\mu(n) = -1$ if n is a square-free positive integer with an odd number of prime factors.**
- **$\mu(n) = 0$ if n has a squared prime factor.**

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def mobius(n):
```

```
    if n == 1:
```

```
        return 1
```

```
    factors = 0
```

```
i = 2
```

```
while i * i <= n:
```

```
    if n % i == 0:
```

```
        factors += 1
```

```
        n //= i
```

```
    if n % i == 0:
```

```
        return 0
```

```
    else:
```

```
        i += 1
```

```
if n > 1:
```

```
    factors += 1
```

```
if factors % 2 == 0:
```

```
    return 1
```

```
else:
```

```
    return -1
```

```
try:
```

```
    num = int(input("Enter a positive integer: "))
```

```
    start_time = time.time()
```

```
    result = mobius(num)
```

```
    end_time = time.time()
```

```
    print("Mobius value:", result)
```

```
    print("Time taken:", end_time - start_time, "seconds")
```

except ValueError:

```
print("Please enter a valid integer")
```

RESULTS ACHIEVED:

Enter a positive integer: 6

Mobius value: 1

Time taken: 9.775161743164062e-06 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 3**Date:** _____**TITLE:**

Week 1, Question 3

AIM/OBJECTIVE(s):

Write a function called `divisor_sum(n)` that calculates the sum of all positive divisors of `n` (including 1 and `n` itself). This is often denoted by $\sigma(n)$.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def divisor_sum(n):
```

```
    total = 0
```

```
    for i in range(1, n + 1):
```

```
        if n % i == 0:
```

```
            total += i
```

```
    return total
```

```
try:
```

```
    number = int(input("Enter a number: "))
```

```
start_time = time.time()
result = divisor_sum(number)
end_time = time.time()

print(f"The sum of divisors of {number} is: {result}")
print(f"Time taken: {end_time - start_time} seconds")

except ValueError:
    print("Please enter a valid integer")
```

RESULTS ACHIEVED:

Enter a number: 5

The sum of divisors of 5 is: 6

Time taken: 1.6927719116210938e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 4**Date:** _____**TITLE:**

Week 1, Question 4

AIM/OBJECTIVE(s):

Write a function called `prime_pi(n)` that approximates the prime-counting function, `n(n)`. This function returns the number of prime numbers less than

or equal to `n`.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import math
```

```
def prime_pi(n: int) -> int:
```

```
    if not isinstance(n, int) or n < 0:
```

```
        raise ValueError("Input 'n' must be a non-negative integer.")
```

```
if n < 2:
```

```
    return 0
```

```
is_prime = [True] * (n + 1)
```

```
is_prime[0] = is_prime[1] = False
```

```
limit = int(math.sqrt(n))
```

```
for p in range(2, limit + 1):
```

```
    if is_prime[p]:
```

```
        for j in range(p * p, n + 1, p):
```

```
            is_prime[j] = False
```

```
return sum(is_prime)
```

```
print(f'π(10) = {prime_pi(10)}') # Expected: 4
```

```
print(f'π(100) = {prime_pi(100)}') # Expected: 25
```

RESULTS ACHIEVED:

$$\pi(10) = 4$$

$$\pi(100) = 25$$

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 5**Date:** _____**TITLE:**

Week 1, Question 5

AIM/OBJECTIVE(s):

Write a function called `legendre_symbol(a, p)` that calculates the Legendre symbol (a/p) , which is a useful function in quadratic reciprocity. It is defined for an odd prime p and an integer a not divisible by p as:

- $(a/p) = 1$ if a is a quadratic residue modulo p (i.e., there exists an integer x such that $x^2 = a \pmod{p}$).
- $(a/p) = -1$ if a is a quadratic non-residue modulo p .

You can calculate it using Euler's criterion: $(a/p) = a^{((p-1)/2)} \pmod{p}$.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def legendre_symbol(a, p):
```

```
    if a % p == 0:
```

```
        return 0
```

```
    exponent = (p - 1) // 2
```

```
    val = pow(a, exponent, p)
```

```
if val == p - 1:
    return -1
return val

try:
    a = int(input("Enter integer a: "))
    p = int(input("Enter odd prime p: "))

    start_time = time.time()
    result = legendre_symbol(a, p)
    end_time = time.time()

    print("Legendre Symbol:", result)
    print("Time taken:", end_time - start_time, "seconds")

except ValueError:
    print("Please enter valid integers")
```

RESULTS ACHIEVED:

Enter integer a: 9

Enter odd prime p: 5

Legendre Symbol: 1

Time taken: 5.91278076171875e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 6**Date:** _____**TITLE:**

Week 2, Question 6

AIM/OBJECTIVE(s):

Write a function factorial(n) that calculates the factorial of a non-negative integer n (n!).

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def factorial(n):
```

```
    if n < 0:
```

```
        return "Error: Number must be non-negative"
```

```
    if n == 0:
```

```
        return 1
```

```
    result = 1
```

```
    for i in range(1, n + 1):
```

```
        result = result * i
```

```
    return result
```

try:

```
num = int(input("Enter a non-negative integer: "))
```

```
start_time = time.time()
```

```
answer = factorial(num)
```

```
end_time = time.time()
```

```
print("The factorial of", num, "is", answer)
```

```
print("Time taken:", end_time - start_time, "seconds")
```

except ValueError:

```
print("Please enter a valid integer")
```

RESULTS ACHIEVED:

Enter a non-negative integer: 7

The factorial of 7 is 5040

Time taken: 2.47955322265625e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 7**Date:** _____**TITLE:**

Week 2, Question 7

AIM/OBJECTIVE(s):

Write a function `is_palindrome(n)` that checks if a number reads the same forwards and backwards.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
def is_palindrome(n):
```

```
    s = str(n)
```

```
    if s == s[::-1]:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
number = int(input("Enter a number: "))
```

```
print(is_palindrome(number))
```

RESULTS ACHIEVED:

Enter a number: 14

False

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 8

Date: _____

TITLE:

Week 2, Question 8

AIM/OBJECTIVE(s):

Write a function `mean_of_digits(n)` that returns the average of all digits in a number.

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def mean_of_digits(n):
```

```
    digits = str(n)
```

```
    total = 0
```

```
    length = 0
```

```
    for d in digits:
```

```
        total = total + int(d)
```

```
        length = length + 1
```

```
    return total / length
```

```
num = int(input("Enter a number: "))

start_time = time.time()
result = mean_of_digits(num)
end_time = time.time()

print("Average of digits:", result)
print("Runtime:", end_time - start_time)
```

RESULTS ACHIEVED:

Enter a number: 55

Average of digits: 5.0

Runtime: 1.8835067749023438e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 9**Date:** _____**TITLE:**

Week 2, Question 9

AIM/OBJECTIVE(s):

Write a function `digital_root(n)` that repeatedly sums the digits of a number until a single digit is obtained.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def digital_root(n):
```

```
    while len(str(n)) > 1:
```

```
        new_sum = 0
```

```
number_as_string = str(n)

for digit_char in number_as_string:

    new_sum = new_sum + int(digit_char)

n = new_sum

return n

number1 = 493193
start_time1 = time.time()
result1 = digital_root(number1)
end_time1 = time.time()

runtime1 = end_time1 - start_time1
print(f"The digital root of {number1} is: {result1}")
print(f"Runtime for {number1}: {runtime1:.6f} seconds")

print("-" * 30)

number2 = 942
start_time2 = time.time()
result2 = digital_root(number2)
end_time2 = time.time()
```

```
runtime2 = end_time2 - start_time2  
print(f"The digital root of {number2} is: {result2}")  
print(f"Runtime for {number2}: {runtime2:.6f} seconds")
```

RESULTS ACHIEVED:

The digital root of 493193 is: 2
Runtime for 493193: 0.000022 seconds

The digital root of 942 is: 6
Runtime for 942: 0.000008 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 10**Date:** _____**TITLE:**

Week 2, Question 10

AIM/OBJECTIVE(s):

Write a function `is_abundant(n)` that returns `True` if the sum of proper divisors of `n` is greater than `n`.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
def is_abundant(n):  
    total = 0  
    for i in range(1, n):  
        if n % i == 0:  
            total = total + i  
  
    if total > n:  
        return True  
    else:  
        return False
```

```
number = 12
```

```
result = is_abundant(number)
print(f'Is {number} abundant? {result}')
```

```
number = 10
result = is_abundant(number)
print(f'Is {number} abundant? {result}')
```

RESULTS ACHIEVED:

Is 12 abundant? True

Is 10 abundant? False

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 11**Date:** _____**TITLE:**

Week 3, Question 11

AIM/OBJECTIVE(s):

Write a function `is_deficient(n)` that returns `True` if the sum of proper divisors of `n` is less than `n`.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def is_deficient(n):
```

```
    total = 0
```

```
    for i in range(1, n):
```

```
        if n % i == 0:
```

```
            total = total + i
```

```
    if total < n:
```

```
        return True
```

```
    else:
```

```
        return False
```

```
n = int(input("Enter a number: "))

start_time = time.time()
result = is_deficient(n)
end_time = time.time()

print("Is the number deficient?", result)
print("Time taken:", end_time - start_time)
```

RESULTS ACHIEVED:

Enter a number: 2
Is the number deficient? True
Time taken: 2.5033950805664062e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 12**Date:** _____**TITLE:**

Week 3, Question 12

AIM/OBJECTIVE(s):

Write a function for harshad number is_harshad(n) that checks if a number is divisible by the sum of its digits.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
def is_harshad(n):  
    if n <= 0:  
        return False  
  
    digits_sum = 0  
    for char in str(n):  
        digits_sum += int(char)  
  
    if n % digits_sum == 0:  
        return True  
    else:  
        return False
```

```
number = int(input("Enter a number: "))

if is_harshad(number):
    print(number, "is a Harshad number")
else:
    print(number, "is not a Harshad number")
```

RESULTS ACHIEVED:

Enter a number: 74

74 is not a Harshad number

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 13**Date:** _____**TITLE:**

Week 3, Question 13

AIM/OBJECTIVE(s):

Write a function `is_automorphic(n)` that checks if a number's square ends with the number itself.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def is_automorphic(n):
```

```
    sq = n * n
```

```
    original = str(n)
```

```
    square = str(sq)
```

```
    if square.endswith(original):
```

```
        return True
```

```
    else:
```

```
        return False
```

```
num = int(input("Enter a number to check: "))

start = time.time()
result = is_automorphic(num)
end = time.time()

if result:
    print(num, "is an automorphic number")
else:
    print(num, "is not an automorphic number")

print("Run time:", end - start)
```

RESULTS ACHIEVED:

Enter a number to check: 6

6 is an automorphic number

Run time: 2.0742416381835938e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 14

Date: _____

TITLE:

Week 3, Question 14

AIM/OBJECTIVE(s):

Write a function `is_pronic(n)` that checks if a number is the product of two consecutive integers.

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def is_pronic(n):  
    if n < 0:  
        return False  
    i = 0  
    while i * (i + 1) <= n:  
        if i * (i + 1) == n:  
            return True  
        i = i + 1  
    return False
```

```
num = int(input("Enter a number: "))
```

```
start = time.time()
```

```
result = is_pronic(num)
```

```
end = time.time()
```

```
if result:
```

```
    print("True")
```

```
else:
```

```
    print("False")
```

```
print("Run time:", end - start)
```

RESULTS ACHIEVED:

Enter a number: 53

False

Run time: 8.821487426757812e-06

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 15**Date:** _____**TITLE:**

Week 3, Question 15

AIM/OBJECTIVE(s):

Write a function `prime_factors(n)` that returns the list of prime factors of a number.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def prime_factors(n):  
    factors = []  
    d = 2  
    while d * d <= n:  
        while n % d == 0:  
            factors.append(d)  
            n //= d  
        d += 1  
    if n > 1:  
        factors.append(n)
```

return factors

```
number = int(input("Enter a number: "))
```

```
start = time.time()
```

```
result = prime_factors(number)
```

```
end = time.time()
```

```
print(result)
```

```
print("Time taken:", end - start)
```

RESULTS ACHIEVED:

Enter a number: 66

[2, 3, 11]

Time taken: 1.52587890625e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 16**Date:** _____**TITLE:**

Week 4, Question 16

AIM/OBJECTIVE(s):

Write a function `count_distinct_prime_factors(n)` that returns how many unique prime factors a number has.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def count_distinct_prime_factors(n):
```

```
    count = 0
```

```
    d = 2
```

```
    temp = n
```

```
    while d * d <= temp:
```

```
        if temp % d == 0:
```

```
            count = count + 1
```

```
            while temp % d == 0:
```

```
                temp = temp // d
```

```
d = d + 1

if temp > 1:
    count = count + 1

return count

number = int(input("Enter a number: "))

start = time.time()
result = count_distinct_prime_factors(number)
end = time.time()

print("Number of distinct prime factors:", result)
print("Time taken:", end - start, "seconds")
```

RESULTS ACHIEVED:

Enter a number: 34

Number of distinct prime factors: 2

Time taken: 1.1205673217773438e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 17

Date: _____

TITLE:

Week 4, Question 17

AIM/OBJECTIVE(s):

Write a function `is_prime_power(n)` that checks if a number can be expressed as p^k where p is prime and $k \geq 1$.

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def is_prime_power(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    p = 0
```

```
    for i in range(2, n + 1):
```

```
        if n % i == 0:
```

```
            p = i
```

```
            break
```

```
while n > 1:
    if n % p != 0:
        return False
    n = n // p

return True

start_time = time.time()

print(is_prime_power(27))
print(is_prime_power(10))
print(is_prime_power(13))
print(is_prime_power(32))

end_time = time.time()
print("Run time:", end_time - start_time)
```

RESULTS ACHIEVED:

True

False

True

True

Run time: 0.00028228759765625

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 18

Date: _____

TITLE:

Week 4, Question 18

AIM/OBJECTIVE(s):

Write a function `is_mersenne_prime(p)` that checks if $2^p - 1$ is a prime number (given that p is prime).

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
import math
```

```
def is_prime(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    for i in range(2, int(math.sqrt(n)) + 1):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

```
def is_mersenne_prime(p):
```

```
mersenne_number = 2 ** p - 1
if is_prime(mersenne_number):
    print(f'2^{p} - 1 is a Mersenne Prime')
    return True
else:
    print(f'2^{p} - 1 is NOT a Mersenne Prime')
    return False

p = 17

start_time = time.time()
is_mersenne_prime(p)
end_time = time.time()

print(f'Time taken: {end_time - start_time} seconds')
```

RESULTS ACHIEVED:

$2^{17} - 1$ is a Mersenne Prime

Time taken: 0.0003781318664550781 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 19**Date:** _____**TITLE:**

Week 4, Question 19

AIM/OBJECTIVE(s):

Write a function `twin_primes(limit)` that generates all twin prime pairs up to a given limit.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def is_prime(num):
```

```
    if num <= 1:
```

```
        return False
```

```
    i = 2
```

```
    while i < num:
```

```
        if num % i == 0:
```

```
            return False
```

```
        i = i + 1
```

```
    return True
```

```
def twin_primes(limit):  
    pairs = []  
    n = 2  
    while n + 2 <= limit:  
        if is_prime(n):  
            if is_prime(n + 2):  
                pair = (n, n + 2)  
                pairs.append(pair)  
        n = n + 1  
    return pairs
```

```
start_time = time.time()  
result = twin_primes(2000)  
end_time = time.time()  
  
print(result)
```

```
elapsed_time = end_time - start_time  
print("Runtime:")  
print(elapsed_time)
```

RESULTS ACHIEVED:

```
[(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43), (59, 61), (71, 73), (101,  
103), (107, 109), (137, 139), (149, 151), (179, 181), (191, 193), (197,  
199), (227, 229), (239, 241), (269, 271), (281, 283), (311, 313), (347,
```

349), (419, 421), (431, 433), (461, 463), (521, 523), (569, 571), (599, 601), (617, 619), (641, 643), (659, 661), (809, 811), (821, 823), (827, 829), (857, 859), (881, 883), (1019, 1021), (1031, 1033), (1049, 1051), (1061, 1063), (1091, 1093), (1151, 1153), (1229, 1231), (1277, 1279), (1289, 1291), (1301, 1303), (1319, 1321), (1427, 1429), (1451, 1453), (1481, 1483), (1487, 1489), (1607, 1609), (1619, 1621), (1667, 1669), (1697, 1699), (1721, 1723), (1787, 1789), (1871, 1873), (1877, 1879), (1931, 1933), (1949, 1951), (1997, 1999)]

Runtime:

0.020779848098754883

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 20**Date:** _____**TITLE:**

Week 4, Question 20

AIM/OBJECTIVE(s):

Write a function Number of Divisors
(d(n)) count_divisors(n) that returns how many positive
divisors a number has.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def count_divisors(n):
```

```
    count = 0
```

```
    for i in range(1, n + 1):
```

```
        if n % i == 0:
```

```
            count += 1
```

```
    return count
```

```
n = int(input("Enter a number: "))
```

```
start = time.time()
result = count_divisors(n)
end = time.time()

print(f'Number of divisors: {result}')
print(f'Time taken: {end - start} seconds')
```

RESULTS ACHIEVED:

Enter a number: 69

Number of divisors: 4

Time taken: 2.2411346435546875e-05 seconds

PS C:\Users\HP>

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 21**Date:** _____**TITLE:**

Week 5, Question 21

AIM/OBJECTIVE(s):

Write a function aliquot_sum(n) that returns the sum of all proper divisors of n (divisors less than n).

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
def aliquot_sum(n):
```

```
    total = 0
```

```
    for i in range(1, n):
```

```
        if n % i == 0:
```

```
            total = total + i
```

```
    return total
```

```
if __name__ == "__main__":
```

```
    number = 12
```

```
    result = aliquot_sum(number)
```

```
    print(f"The aliquot sum of {number} is {result}")
```

```
number = 10  
result = aliquot_sum(number)  
print(f"The aliquot sum of {number} is {result}")
```

RESULTS ACHIEVED:

The aliquot sum of 12 is 16

The aliquot sum of 10 is 8

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 22**Date:** _____**TITLE:**

Week 5, Question 22

AIM/OBJECTIVE(s):

Write a function are_amicable(a, b) that checks if two numbers are amicable (sum of proper divisors of a equals b and vice versa).

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def sum_divisors(n):
```

```
    total = 0
```

```
    for i in range(1, n):
```

```
        if n % i == 0:
```

```
            total = total + i
```

```
    return total
```

```
def are_amicable(a, b):
```

```
    sum_a = sum_divisors(a)
```

```
sum_b = sum_divisors(b)

if sum_a == b and sum_b == a:
    return True
else:
    return False

num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

start_time = time.time()

result = are_amicable(num1, num2)

end_time = time.time()

if result:
    print("The numbers are amicable")
else:
    print("The numbers are not amicable")

print("Time taken:", end_time - start_time, "seconds")
```

RESULTS ACHIEVED:

Enter first number: 85

Enter second number: 45

The numbers are not amicable

Time taken: 2.8133392333984375e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 23**Date:** _____**TITLE:**

Week 5, Question 23

AIM/OBJECTIVE(s):

Write a function `multiplicative_persistence(n)` that counts how many steps until a number's digits multiply to a single digit.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def multiplicative_persistence(n):
```

```
    steps = 0
```

```
    while n >= 10:
```

```
        product = 1
```

```
        for digit in str(n):
```

```
            product *= int(digit)
```

```
        n = product
```

```
        steps += 1
```

```
    return steps
```

```
if __name__ == "__main__":  
    try:  
        num = int(input("Enter a number: "))  
  
        start_time = time.time()  
        result = multiplicative_persistence(num)  
        end_time = time.time()  
  
        print(f'Multiplicative Persistence: {result}')  
        print(f'Run time: {end_time - start_time} seconds')  
  
    except ValueError:  
        print("Please enter a valid integer")
```

RESULTS ACHIEVED:

Enter a number: 5

Multiplicative Persistence: 0

Run time: 4.0531158447265625e-06 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 24

Date: _____

TITLE:

Week 5, Question 24

AIM/OBJECTIVE(s):

Write a function `is_highly_composite(n)` that checks if a number has more divisors than any smaller number.

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def count_divisors(num):  
    count = 0  
    for i in range(1, num + 1):  
        if num % i == 0:  
            count = count + 1  
    return count
```

```
def is_highly_composite(n):  
    n_divisors = count_divisors(n)
```

```
for i in range(1, n):  
    i_divisors = count_divisors(i)  
    if i_divisors >= n_divisors:  
        return False  
  
return True  
  
num = int(input("Enter a number: "))  
  
start = time.time()  
result = is_highly_composite(num)  
end = time.time()  
  
print(result)  
print("Run time:", end - start)
```

RESULTS ACHIEVED:

Enter a number: 77

False

Run time: 2.5510787963867188e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 25

Date: _____

TITLE:

Week 5, Question 25

AIM/OBJECTIVE(s):

**Write a function for Modular
Exponentiation mod_exp(base, exponent, modulus) that
efficiently calculates (base^{exponent}) % modulus.**

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def mod_exp(base, exponent, modulus):  
    result = 1  
    base = base % modulus  
    while exponent > 0:  
        if exponent % 2 == 1:  
            result = (result * base) % modulus  
        exponent = exponent // 2  
        base = (base * base) % modulus  
    return result
```

```
if __name__ == "__main__":  
    b = int(input("Enter base: "))  
    e = int(input("Enter exponent: "))  
    m = int(input("Enter modulus: "))  
  
    start_time = time.time()  
    ans = mod_exp(b, e, m)  
    end_time = time.time()  
  
    print("Result:", ans)  
    print("Time taken:", end_time - start_time, "seconds")
```

RESULTS ACHIEVED:

Enter base: 6

Enter exponent: 8

Enter modulus: 45

Result: 36

Time taken: 2.1219253540039062e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 26**Date:** _____**TITLE:**

Week 6, Question 26

AIM/OBJECTIVE(s):

**Write a function Modular Multiplicative
Inverse mod_inverse(a, m) that finds the number x such
that $(a * x) = 1 \bmod m$.**

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def mod_inverse(a, m):  
    for x in range(1, m):  
        if ((a * x) % m == 1):  
            return x  
    return -1
```

```
try:  
    a_val = int(input("Enter a: "))  
    m_val = int(input("Enter m: "))
```

```
start_time = time.time()
result = mod_inverse(a_val, m_val)
end_time = time.time()

print("Modular Multiplicative Inverse is:", result)
print("Time taken:", end_time - start_time, "seconds")

except ValueError:
    print("Please enter valid integers")
```

RESULTS ACHIEVED:

Enter a: 1

Enter m: 9

Modular Multiplicative Inverse is: 1

Time taken: 1.1444091796875e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 27

Date: _____

TITLE:

Week 6, Question 27

AIM/OBJECTIVE(s):

Write a function chinese Remainder Theorem Solver crt(remainders, moduli) that solves a system of congruences $x = r; \text{ mod } m_i$.

METHODOLOGY & TOOL USED:

Python and Visual studio code

BRIEF DESCRIPTION:

```
import time
```

```
def find_modular_inverse(a, m):
```

```
    m0 = m
```

```
    y = 0
```

```
    x = 1
```

```
    if m == 1:
```

```
        return 0
```

```
    while a > 1:
```

$q = a // m$

$t = m$

$m = a \% m$

$a = t$

$t = y$

$y = x - q * y$

$x = t$

if $x < 0$:

$x = x + m0$

return x

def crt(remainders, moduli):

total_product = 1

for x in moduli:

total_product = total_product * x

result = 0

for i in range(len(moduli)):

current_mod = moduli[i]

current_remainder = remainders[i]

partial_product = total_product // current_mod

inverse = find_modular_inverse(partial_product, current_mod)

result = result + (current_remainder * partial_product * inverse)

```
return result % total_product

if __name__ == "__main__":
    a = [2, 3, 2]
    m = [3, 5, 7]

    start_time = time.time()

    solution = crt(a, m)

    end_time = time.time()

    print("System of congruences:")
    for i in range(len(a)):
        print("x =", a[i], "mod", m[i])

    print("\nSolution x =", solution)
    print("Runtime:", end_time - start_time, "seconds")
```

RESULTS ACHIEVED:

System of congruences:

$$x = 2 \pmod{3}$$

$$x = 3 \pmod{5}$$

$$x = 2 \pmod{7}$$

Solution $x = 23$

Runtime: 1.239776611328125e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 28**Date:** _____**TITLE:**

Week 6, Question 28

AIM/OBJECTIVE(s):

Write a function Quadratic Residue
Check `is_quadratic_residue(a, p)` that checks if $x^2 = a \pmod p$
has a solution.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def is_quadratic_residue(a, p):
```

```
    for x in range(p):
```

```
        if (x * x) % p == a:
```

```
            return True
```

```
    return False
```

```
if __name__ == "__main__":
```

```
    a = 2
```

```
    p = 7
```

```
start = time.time()
result = is_quadratic_residue(a, p)
end = time.time()

print(result)
print(f"Time taken: {end - start} seconds")
```

RESULTS ACHIEVED:

True

Time taken: 7.152557373046875e-06 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 29**Date:** _____**TITLE:**

Week 6, Question 29

AIM/OBJECTIVE(s):

Write a function `order_mod(a, n)` that finds the smallest positive integer `k` such that $ak = 1 \pmod n$.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**`import time``import math``def order_mod(a, n):` `if math.gcd(a, n) != 1:` `return -1` `k = 1` `current = a % n` `while current != 1:` `current = (current * a) % n`

```
k += 1

return k

a = 3
n = 7

start_time = time.time()
result = order_mod(a, n)
end_time = time.time()

print(f"The order is: {result}")
print(f"Run time: {end_time - start_time} seconds")
```

RESULTS ACHIEVED:

The order is: 6

Run time: 3.814697265625e-06 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 30**Date:** _____**TITLE:**

Week 6, Question 30

AIM/OBJECTIVE(s):

Write a function Fibonacci Prime Check is_fibonacci_prime(n) that checks if a number is both Fibonacci and prime.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**`import time``import math``def is_prime(n):` `if n <= 1:` `return False` `for i in range(2, int(n**0.5) + 1):` `if n % i == 0:` `return False` `return True`

```
def is_fibonacci(n):  
    if n < 0:  
        return False  
    a = 0  
    b = 1  
    while a < n:  
        temp = a  
        a = b  
        b = temp + b  
    return a == n  
  
def is_fibonacci_prime(n):  
    if is_fibonacci(n) and is_prime(n):  
        return True  
    return False  
  
if __name__ == "__main__":  
    try:  
        num = int(input("Enter a number to check: "))  
  
        start_time = time.time()  
        result = is_fibonacci_prime(num)  
        end_time = time.time()  
  
        if result:  
            print(f'{num} is a Fibonacci Prime number.')  
        else:  
            print(f'{num} is NOT a Fibonacci Prime number.')
```

```
print(f'Execution time: {end_time - start_time:.6f} seconds')  
  
except ValueError:  
    print("Please enter a valid integer.")
```

RESULTS ACHIEVED:

Enter a number to check: 17

17 is NOT a Fibonacci Prime number.

Execution time: 0.000008 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 31**Date:** _____**TITLE:**

Week 7, Question 31

AIM/OBJECTIVE(s):

Write a function Lucas Numbers
Generator `lucas_sequence(n)` that generates the first n
Lucas numbers (similar to Fibonacci but starts with 2,
1).

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def lucas_sequence(n):
```

```
    lucas_list = []
```

```
    if n >= 1:
```

```
        lucas_list.append(2)
```

```
    if n >= 2:
```

```
        lucas_list.append(1)
```

```
for i in range(2, n):
    next_num = lucas_list[i-1] + lucas_list[i-2]
    lucas_list.append(next_num)

return lucas_list

n = int(input("Enter the number of terms: "))

start = time.time()
result = lucas_sequence(n)
end = time.time()

print("Lucas Sequence:", result)
print("Time taken:", end - start)
```

RESULTS ACHIEVED:

Enter the number of terms: 43

Lucas Sequence: [2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, 521, 843, 1364, 2207, 3571, 5778, 9349, 15127, 24476, 39603, 64079, 103682, 167761, 271443, 439204, 710647, 1149851, 1860498, 3010349, 4870847, 7881196, 12752043, 20633239, 33385282, 54018521, 87403803, 141422324, 228826127, 370248451, 599074578]

Time taken: 5.984306335449219e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 32**Date:** _____**TITLE:**

Week 7, Question 32

AIM/OBJECTIVE(s):**Write a function for Perfect Powers****Check `is_perfect_power(n)` that checks if a number can be expressed as ab where $a > 0$ and $b > 1$.****METHODOLOGY & TOOL USED:****Python and Visual studio code****BRIEF DESCRIPTION:**`import time``def is_perfect_power(n):` `if n <= 1:` `return True` `limit = int(n ** 0.5)` `for a in range(2, limit + 1):` `temp = a * a` `while temp <= n:`

```
        if temp == n:
            return True
        temp = temp * a

    return False

n = int(input("Enter a number: "))

start = time.time()
result = is_perfect_power(n)
end = time.time()

print(result)
print(end - start)
```

RESULTS ACHIEVED:

Enter a number: 8

True

3.314018249511719e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 33**Date:** _____**TITLE:**

Week 7, Question 33

AIM/OBJECTIVE(s):**Write a function Collatz****Sequence Length collatz_length(n) that returns the number of steps for n to reach 1 in the Collatz conjecture.****METHODOLOGY & TOOL USED:****Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def collatz_length(n):  
    steps = 0  
    while n != 1:  
        if n % 2 == 0:  
            n = n // 2  
        else:  
            n = 3 * n + 1  
        steps = steps + 1  
    return steps
```

```
number = int(input("Enter a number: "))

start = time.time()
result = collatz_length(number)
end = time.time()

print("Steps:", result)
print("Time taken:", end - start, "seconds")
```

RESULTS ACHIEVED:

Enter a number: 5

Steps: 5

Time taken: 1.1682510375976562e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 34**Date:** _____**TITLE:**

Week 7, Question 34

AIM/OBJECTIVE(s):

Write a function Polygonal Numbers `polygonal_number(s, n)` that returns the n-th s-gonal number.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def polygonal_number(s, n):
```

```
    part1 = (s - 2) * n * n
```

```
    part2 = (s - 4) * n
```

```
    result = (part1 - part2) // 2
```

```
    return result
```

```
s_input = int(input("Enter the value of s (sides): "))
```

```
n_input = int(input("Enter the value of n (position): "))
```

```
start_time = time.time()
```

```
answer = polygonal_number(s_input, n_input)  
end_time = time.time()
```

```
print("The polygonal number is:")  
print(answer)  
print("Time taken to run:")  
print(end_time - start_time)
```

RESULTS ACHIEVED:

Enter the value of s (sides): 8

Enter the value of n (position): 4

The polygonal number is:

40

Time taken to run:

1.3828277587890625e-05

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 35**Date:** _____**TITLE:**

Week 7, Question 35

AIM/OBJECTIVE(s):**Write a function Carmichael Number****Check is_carmichael(n) that checks if a composite number n satisfies $a^{n-1} \equiv 1 \pmod n$ for all a coprime to n.****METHODOLOGY & TOOL USED:****Python and Visual studio code****BRIEF DESCRIPTION:**`import math``import time``def is_prime(num):` `if num < 2:` `return False` `for i in range(2, int(num ** 0.5) + 1):` `if num % i == 0:` `return False` `return True`

```
def is_carmichael(n):  
    if is_prime(n):  
        return False  
  
    for a in range(2, n):  
        if math.gcd(a, n) == 1:  
            if pow(a, n - 1, n) != 1:  
                return False  
    return True  
  
number_to_check = int(input("Enter a number: "))  
  
start_time = time.time()  
result = is_carmichael(number_to_check)  
end_time = time.time()  
  
print(f'Is {number_to_check} a Carmichael number? {result}')  
print(f'Time taken: {end_time - start_time} seconds')
```

RESULTS ACHIEVED:

Enter a number: 12

Is 12 a Carmichael number? False

Time taken: 0.0007123947143554688 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 36**Date:** _____**TITLE:**

Week 8, Question 36

AIM/OBJECTIVE(s):

**Implement the probabilistic
Miller-Rabin test is_prime_miller_rabin(n, k) with k rounds.**

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import random
```

```
import time
```

```
def is_prime_miller_rabin(n, k):
```

```
    if n <= 1:
```

```
        return False
```

```
    if n <= 3:
```

```
        return True
```

```
    if n % 2 == 0:
```

```
        return False
```

```
    r = 0
```

```
d = n - 1
```

```
while d % 2 == 0:
```

```
    r += 1
```

```
    d //= 2
```

```
for _ in range(k):
```

```
    a = random.randint(2, n - 2)
```

```
    x = pow(a, d, n)
```

```
    if x == 1 or x == n - 1:
```

```
        continue
```

```
is_composite = True
```

```
for _ in range(r - 1):
```

```
    x = pow(x, 2, n)
```

```
    if x == n - 1:
```

```
        is_composite = False
```

```
        break
```

```
if is_composite:
```

```
    return False
```

```
return True
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        num = int(input("Enter number to test: "))
```

```
        rounds = int(input("Enter number of rounds (k): "))
```

```
start_time = time.time()

result = is_prime_miller_rabin(num, rounds)

end_time = time.time()

print("Result:", result)

print("Time taken:", end_time - start_time, "seconds")

except ValueError:

    print("Please enter valid integers")
```

RESULTS ACHIEVED:

Enter number to test: 9

Enter number of rounds (k): 5

Result: False

Time taken: 5.14984130859375e-05 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 37**Date:** _____**TITLE:**

Week 8, Question 37

AIM/OBJECTIVE(s):

Implement `pollard_rho(n)` for integer factorization using Pollard's rho algorithm.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import random
```

```
import time
```

```
def gcd(a, b):
```

```
    while b:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def pollard_rho(n):
```

```
    if n % 2 == 0:
```

```
        return 2
```

```
x = random.randint(2, n - 1)
```

```
y = x
```

```
c = random.randint(1, n - 1)
```

```
d = 1
```

```
while d == 1:
```

```
    x = (x * x + c) % n
```

```
    y = (y * y + c) % n
```

```
    y = (y * y + c) % n
```

```
    d = gcd(abs(x - y), n)
```

```
    if d == n:
```

```
        return pollard_rho(n)
```

```
return d
```

```
if __name__ == "__main__":
```

```
    try:
```

```
        num = int(input("Enter an integer to factorize: "))
```

```
        start_time = time.time()
```

```
        factor = pollard_rho(num)
```

```
        end_time = time.time()
```

```
        print("One factor is:", factor)
```

```
        print("Time taken:", end_time - start_time, "seconds")
```

```
    except ValueError:
```

```
print("Please enter a valid integer")
```

RESULTS ACHIEVED:

Enter an integer to factorize: 74

One factor is: 2

Time taken: 5.0067901611328125e-06 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 38**Date:** _____**TITLE:**

Week 8, Question 38

AIM/OBJECTIVE(s):

Write a function `zeta_approx(s, terms)` that approximates the Riemann zeta function $\zeta(s)$ using the first 'terms' of the series.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def zeta_approx(s, terms):  
    total = 0  
    for i in range(1, terms + 1):  
        total = total + (1 / (i ** s))  
    return total
```

```
s_input = 2
```

```
n_terms = 100000
```

```
start = time.time()
result = zeta_approx(s_input, n_terms)
end = time.time()

print("Result:", result)
print("Time taken:", end - start, "seconds")
```

RESULTS ACHIEVED:

Result: 1.6449240668982423

Time taken: 0.007271766662597656 seconds

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure

Practical No: 39**Date:** _____**TITLE:**

Week 8, Question 39

AIM/OBJECTIVE(s):

Write a function Partition Function
p(n) partition_function(n) that calculates the number of
distinct ways to write n as a sum of positive integers.

METHODOLOGY & TOOL USED:**Python and Visual studio code****BRIEF DESCRIPTION:**

```
import time
```

```
def partition_function(n):
```

```
    start_time = time.time()
```

```
    ways = [0] * (n + 1)
```

```
    ways[0] = 1
```

```
    for i in range(1, n + 1):
```

```
        for j in range(i, n + 1):
```

```
            ways[j] += ways[j - i]
```

```
end_time = time.time()
print("Time taken:", end_time - start_time)
```

```
return ways[n]
```

```
number = int(input("Enter a number: "))
result = partition_function(number)
print("Distinct ways:", result)
```

RESULTS ACHIEVED:

Enter a number: 82

Time taken: 0.002022981643676758

Distinct ways: 20506255

DIFFICULTY FACED BY STUDENT:

None

SKILLS ACHIEVED:

Dynamic Programming (DP)

List Manipulation

Performance Measurement

Function Structure